

Data Engineer Interview — Full Answer Key

Companion to the interview roadmap. Answers are written concise and interview-ready — practice saying them out loud, then extend with your own project examples where noted.

ROUND 1: Online Assessment — SQL Questions

1. Second highest salary per department

```
sql

SELECT department_id, employee_id, salary
FROM (
    SELECT department_id, employee_id, salary,
           DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS
    FROM employees
) t
WHERE rnk = 2;
```

Use `DENSE_RANK()` so ties don't skip a rank — two people tied for highest still make "second highest" the next distinct value.

2. Employees who earn more than their manager

```
sql

SELECT e.employee_id, e.name
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;
```

This is a self-join — the same table plays two roles (employee and manager).

3. Running total / moving average of transactions

```
sql

SELECT txn_date, amount,
       SUM(amount) OVER (ORDER BY txn_date) AS running_total,
       AVG(amount) OVER (ORDER BY txn_date ROWS BETWEEN 6 PRECEDING AND CURRENT
FROM transactions;
```

4. Find duplicate records

```
sql
```

```
SELECT customer_id, txn_date, amount, COUNT(*)
FROM transactions
GROUP BY customer_id, txn_date, amount
HAVING COUNT(*) > 1;
```

5. Nth highest value without LIMIT/TOP

```
sql

SELECT DISTINCT salary
FROM employees e1
WHERE N-1 = (
    SELECT COUNT(DISTINCT salary) FROM employees e2 WHERE e2.salary > e1.salary
);
```

Or use `DENSE_RANK()` as in Q1 and filter `WHERE rnk = N` — cleaner and preferred in interviews.

6. Gaps and islands (continuous date ranges)

```
sql

SELECT MIN(txn_date) AS island_start, MAX(txn_date) AS island_end
FROM (
    SELECT txn_date,
           DATE_SUB(txn_date, INTERVAL ROW_NUMBER() OVER (ORDER BY txn_date) DAY)
    FROM daily_activity
) t
GROUP BY grp;
```

Trick: subtracting a running row number from the date turns consecutive dates into the same constant value (`grp`) — group by it to find islands.

7. Rank customers by spend

```
sql

SELECT customer_id, SUM(amount) AS total_spend,
       RANK() OVER (ORDER BY SUM(amount) DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY SUM(amount) DESC) AS dense_rnk,
       ROW_NUMBER() OVER (ORDER BY SUM(amount) DESC) AS row_num
FROM transactions
GROUP BY customer_id;
```

`RANK` skips numbers after a tie (1,1,3), `DENSE_RANK` doesn't (1,1,2), `ROW_NUMBER` never ties (always unique, arbitrary among ties).

ROUND 1: Online Assessment — Python/DSA Questions

1. Reverse a string/list without built-ins

python

```
def reverse_list(arr):
    left, right = 0, len(arr) - 1
    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1
    return arr
```

2. Find missing number(s) in an array

python

```
def find_missing(arr, n):
    expected_sum = n * (n + 1) // 2
    return expected_sum - sum(arr)
```

For multiple missing numbers, use a `set` difference: `set(range(1, n+1)) - set(arr)`.

3. Two sum

python

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
```

$O(n)$ using a hashmap instead of $O(n^2)$ brute force — always mention this tradeoff.

4. Count word frequency in a large file

python

```
from collections import Counter

def word_freq(filepath):
    counter = Counter()
    with open(filepath) as f:
        for line in f:
            # streams line by line, doesn't load whole
            counter.update(line.split())
    return counter
```

Key DE point: never do `f.read()` on a huge file — iterate line by line to keep memory flat.

5. Detect duplicates efficiently

python

```
def has_duplicates(arr):
    return len(arr) != len(set(arr))
```

6. Merge two sorted lists

python

```
def merge_sorted(a, b):
    result, i, j = [], 0, 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i]); i += 1
        else:
            result.append(b[j]); j += 1
    result.extend(a[i:])
    result.extend(b[j:])
    return result
```

7. Read a large CSV in chunks

python

```
import pandas as pd

for chunk in pd.read_csv("big_file.csv", chunksize=100_000):
    process(chunk)  # process each chunk instead of loading everything at once
```

ROUND 2: SQL Deep Dive

1. WHERE vs HAVING `WHERE` filters rows before grouping; `HAVING` filters groups after `GROUP BY`. You can't use an aggregate like `COUNT()` in `WHERE` — that's what `HAVING` is for.

2. INNER vs LEFT vs FULL OUTER JOIN

- **INNER JOIN**: only matching rows in both tables.
- **LEFT JOIN**: all rows from the left table, matched rows from the right (NULLs where no match).
- **FULL OUTER JOIN**: all rows from both, NULLs where no match on either side. Example: joining **orders** (left) with **customers** — a **LEFT JOIN** keeps orders even if the customer record was deleted.

3. Window functions — RANK vs DENSE_RANK vs ROW_NUMBER Covered above (OA Q7). Interview follow-up: "which would you use to remove duplicates, keeping one row per group?" → **ROW_NUMBER()** partitioned by the dedup key, then filter **WHERE row_num = 1**.

4. CTE for hierarchical data (recursive)

```
sql

WITH RECURSIVE org_chart AS (
  SELECT employee_id, manager_id, name, 1 AS level
  FROM employees WHERE manager_id IS NULL          -- anchor: top of hierarchy
  UNION ALL
  SELECT e.employee_id, e.manager_id, e.name, oc.level + 1
  FROM employees e
  JOIN org_chart oc ON e.manager_id = oc.employee_id
)
SELECT * FROM org_chart;
```

5. How indexes work An index is a separate sorted data structure (usually a B-tree) that lets the database find rows without scanning the whole table — like a book's index instead of reading every page. It speeds up reads but adds overhead on every **INSERT**/**UPDATE**/**DELETE** since the index must be updated too.

6. When does an index slow things down?

- On small tables (full scan is already fast).
- On columns with low cardinality (e.g., a boolean column) — the optimizer may ignore the index anyway.
- On tables with heavy write traffic, since every write also updates the index.
- Too many indexes on one table slows down all writes.

7. Clustered vs non-clustered index Clustered: the table data itself is physically sorted by the index key — only one per table. Non-clustered: a separate structure with pointers back to the actual rows — a table can have many.

8. Optimizing a slow query on a 10M-row table

- Check the execution plan for full table scans.
- Add indexes on join/filter columns.

- Avoid `(SELECT *)` — pull only needed columns.
- Avoid functions on indexed columns in `(WHERE)` (e.g., `(WHERE YEAR(date) = 2024)` blocks index use — rewrite as a range filter).
- Consider partitioning the table by date if it's append-heavy.
- Rewrite correlated subqueries as joins where possible.

9. Execution plan A step-by-step breakdown of how the database engine will run your query — shows whether it's doing an index scan or full table scan, join order, and estimated cost. You read it to find the most expensive step and optimize that first.

10. Normalization (1NF, 2NF, 3NF)

- 1NF: atomic values, no repeating groups in a column.
- 2NF: 1NF + no partial dependency (non-key columns depend on the *whole* composite key, not part of it).
- 3NF: 2NF + no transitive dependency (non-key columns depend only on the key, not on other non-key columns). Example: a table with `(order_id, product_id, product_name)` violates 3NF because `(product_name)` depends on `(product_id)`, not on the order — split into a separate `(products)` table.

11. Star vs snowflake schema Star: one central fact table joined directly to denormalized dimension tables (simple, fast reads, some redundancy). Snowflake: dimension tables are further normalized into sub-dimensions (less redundancy, more joins, slightly slower reads). Banks often favor star schema for reporting speed.

12. Top 3 products by revenue per region

```
sql

SELECT region, product, revenue
FROM (
    SELECT region, product, SUM(sales) AS revenue,
           ROW_NUMBER() OVER (PARTITION BY region ORDER BY SUM(sales) DESC) AS rn
    FROM sales
    GROUP BY region, product
) t
WHERE rn <= 3;
```

13. Handling NULLs in aggregations Most aggregate functions (`(SUM)`, `(AVG)`, `(COUNT(column))`) ignore NULLs automatically. `(COUNT(*))` counts all rows including NULLs, `(COUNT(column))` doesn't. Use `(COALESCE(column, 0))` when you need NULL treated as a real value in a calculation.

14. UNION vs UNION ALL `(UNION)` removes duplicate rows across the combined result (slower, does a sort/dedup pass). `(UNION ALL)` keeps everything including duplicates (faster). Use `(UNION ALL)` unless you specifically need deduplication.

ROUND 2: Python Deep Dive

1. List vs tuple vs set vs dict

- List: ordered, mutable, allows duplicates — use for sequences you'll modify.
- Tuple: ordered, immutable — use for fixed collections (e.g., coordinates), also hashable so usable as dict keys.
- Set: unordered, unique elements — use for membership checks and dedup ($O(1)$ lookup vs $O(n)$ for lists).
- Dict: key-value pairs, $O(1)$ lookup by key — use when you need fast access by a label.

2. Generators — yield vs return `return` exits the function and hands back one value; `yield` pauses the function and hands back one value at a time, resuming where it left off on the next call. Generators don't hold the entire result in memory — critical for processing large datasets row by row instead of loading everything at once.

3. List comprehension vs generator expression `[x for x in range(1000000)]` builds the entire list in memory immediately. `(x for x in range(1000000))` creates a generator that produces values lazily, one at a time — use generator expressions when you don't need to store the whole result, just iterate once.

4. Processing a 10GB file without loading it fully Read line by line (files are natively iterable in Python) or use `pandas.read_csv(chunksize=...)`. Never call `.read()` or `.readlines()` on the whole file at once — that loads everything into RAM.

5. deepcopy vs shallow copy Shallow copy (`copy.copy()`) copies the outer object but nested objects still point to the same references — modifying a nested list in the copy affects the original. `deepcopy()` recursively copies everything, so the copy is fully independent.

6. Decorators A decorator wraps a function to add behavior without modifying its code — e.g., logging, timing, or caching. Example:

```
python

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time()-start:.2f}s")
        return result
    return wrapper

@timer
def load_data():
    ...
```

****7. *args and kwargs** (`*args`) collects extra positional arguments into a tuple; (`**kwargs`) collects extra keyword arguments into a dict. Used when you don't know in advance how many arguments a function needs to accept — common in wrapper/decorator functions and flexible pipeline configs.

8. merge() vs join() vs concat() in pandas

- (`merge()`): SQL-style joins on columns (like a database join).
- (`join()`): joins on the index by default (shorthand for merge on index).
- (`concat()`): stacks DataFrames either vertically (append rows) or horizontally (add columns), no key-based matching.

9. Handling missing data in pandas (`df.fillna(value)`) replaces NULLs with a value (mean, median, 0, forward-fill). (`df.dropna()`) removes rows/columns with NULLs.

Interpolation (`df.interpolate()`) estimates missing values based on surrounding data — useful for time series. Choice depends on whether missing data is random or meaningful (e.g., a missing "end date" might mean "still active," not truly missing).

10. apply() vs map() vs applymap()

- (`map()`): element-wise, only on a Series.
- (`apply()`): works on Series or DataFrame, can apply a function along rows or columns.
- (`applymap()`): element-wise, but across the entire DataFrame (deprecated in favor of `.map()` in newer pandas — worth mentioning you know this).

11. Detect duplicate transactions in a DataFrame

```
python
duplicates = df[df.duplicated(subset=['customer_id', 'amount', 'txn_date'], keep=False)]
```

12. GIL (Global Interpreter Lock) A mutex that allows only one thread to execute Python bytecode at a time, even on multi-core machines. This means multithreading doesn't help CPU-bound tasks in Python (no true parallel execution), but it's fine for I/O-bound tasks (waiting on network/disk) since the GIL is released during I/O waits.

13. Parallelizing a data job — multiprocessing vs threading Use (`multiprocessing`) for CPU-bound work (data transformation, heavy computation) since each process gets its own Python interpreter and bypasses the GIL. Use (`threading`) (or (`asyncio`)) for I/O-bound work (API calls, file/network waits) where the bottleneck is waiting, not computing. For genuinely large-scale data processing, the real answer is usually "use Spark," which handles distributed parallelism across a cluster rather than a single machine.

ROUND 3: Data Engineering Concepts

1. ETL vs ELT ETL: transform data *before* loading it into the target system — used when the destination has limited compute or when you need to clean data before storage. ELT: load

raw data first, transform *inside* the destination (e.g., using dbt in a warehouse) — used when the destination (Snowflake/BigQuery/Redshift) has strong compute power and you want the raw data preserved for reprocessing.

2. Designing a multi-source daily ingestion pipeline Clarify volumes and SLAs first. Then: extract from each source (API pagination, DB CDC/full extract, file drops) into a staging/raw zone (e.g., S3), validate schema and row counts, transform into a conformed model, load into the warehouse, and orchestrate the whole thing with retries and alerting (Airflow). Keep raw data immutable so you can reprocess if a transformation bug is found later.

3. Handling schema evolution Use schema registries or explicit schema versioning. For additive changes (new column), default to nullable and backward-compatible reads. For breaking changes (column removed/renamed), version the schema and maintain a mapping layer, or fail the pipeline loudly rather than silently dropping data — silent failures are worse than pipeline crashes in banking data.

4. Ensuring data quality in a pipeline Row count reconciliation (source vs target), null/duplicate checks, referential integrity checks, schema validation, and automated alerts when thresholds are breached (e.g., using Great Expectations or custom checks). Log data quality metrics over time so anomalies are visible, not just pass/fail.

5. Idempotency Running the same pipeline twice on the same input produces the same result, with no duplicate or corrupted data. Achieved via upserts (`(MERGE)`) instead of blind inserts, or by deleting-and-reinserting a partition rather than appending. Critical for retries — if a pipeline fails halfway and reruns, idempotency prevents double-counted transactions.

6. Handling a pipeline failure halfway through Design writes to be atomic at the partition/batch level — write to a staging table/location first, validate, then swap/commit only on success (or use transactional writes if the storage layer supports it, like Delta Lake/Iceberg). Never let partial writes land directly in the production table.

7. Batch vs streaming — how to decide Batch: acceptable latency in minutes/hours, simpler infra, cheaper, good for reporting/reconciliation. Streaming: sub-second/near-real-time requirement, e.g., fraud detection — but adds significant complexity (state management, out-of-order events, exactly-once semantics). Default to batch unless there's a clear business need for real-time.

8. Data lake vs warehouse vs lakehouse Data lake: stores raw, unstructured/structured data cheaply (e.g., S3) — flexible but no strong schema enforcement. Data warehouse: structured, schema-enforced, optimized for fast analytical queries (Redshift/Snowflake) — more expensive, less flexible for raw/unstructured data. Lakehouse: combines both — cheap storage with warehouse-like transactional guarantees and schema enforcement (Delta Lake, Iceberg, Databricks).

9. Handling late-arriving data in a batch job Design pipelines to reprocess a trailing window (e.g., last 3 days) rather than only "today," so late records get picked up.

Alternatively, use a watermark strategy — accept data up to N hours late, and flag/quarantine anything later than that for manual review.

10. Incremental load / CDC Instead of reloading the whole source table each run, track only what changed since the last run — via a `last_updated` timestamp column, database transaction logs (true CDC, e.g., Debezium), or an incrementing ID. Reduces load time and resource usage dramatically on large tables.

ROUND 3: Apache Spark

1. DataFrame vs RDD RDD: low-level, distributed collection of objects, no built-in optimization — you manage everything manually. DataFrame: higher-level, has a schema, and benefits from Spark's Catalyst optimizer and Tungsten execution engine, which generate optimized query plans automatically. DataFrames are preferred today for almost everything except very custom/low-level transformations.

2. Lazy evaluation Spark doesn't execute transformations (`map`, `filter`, `select`) immediately — it builds a logical execution plan (DAG) and only runs it when an action (`count()`, `collect()`, `write()`) is called. This lets Spark optimize the whole chain of operations before running anything, rather than executing step by step.

3. Shuffle A shuffle is when data needs to be redistributed across partitions/nodes — e.g., for a `groupBy` or `join` on a key not already co-located. It's expensive because it involves disk I/O, network transfer, and serialization across the cluster. Minimizing shuffles (via smart partitioning, broadcast joins) is a core Spark performance skill.

4. Handling data skew in a join If one key has disproportionately more rows than others (e.g., one `customer_id` with millions of transactions vs others with a few), that partition becomes a bottleneck. Fixes: salting the skewed key (adding a random suffix to spread it across partitions), or using a broadcast join if one side is small enough to fit in memory.

5. repartition() vs coalesce() `repartition()` does a full shuffle and can increase or decrease partitions, giving even distribution. `coalesce()` only decreases partitions and avoids a full shuffle by merging adjacent partitions — cheaper, but can leave uneven partition sizes.

6. Broadcast join When joining a large table with a small one, Spark can send ("broadcast") a full copy of the small table to every executor, avoiding a shuffle of the large table entirely. Use when one side is small enough to fit comfortably in each executor's memory (typically under ~10MB—a few hundred MB depending on cluster config).

7. Optimizing a slow Spark job Check the Spark UI for skewed stages/shuffles. Use broadcast joins where applicable, cache intermediate DataFrames reused multiple times, choose efficient file formats (Parquet over CSV — columnar, compressed, splittable), and avoid unnecessary wide transformations (`groupBy`, `distinct`) on huge datasets.

8. Narrow vs wide transformations Narrow: each input partition contributes to only one output partition (e.g., `map`, `filter`) — no shuffle needed. Wide: data from multiple

partitions needs to combine into new partitions (e.g., `groupBy`, `join`, `distinct`) — requires a shuffle.

ROUND 3: Orchestration (Airflow)

1. What is a DAG? A Directed Acyclic Graph — a set of tasks with defined dependencies and no circular loops. In Airflow, a DAG defines what runs, in what order, and how often (schedule).

2. Handling a failed task Configure retries with exponential backoff at the task level, set up alerting (email/Slack) on failure, and use Airflow's built-in backfill capability to rerun a specific historical date range once the underlying issue is fixed.

3. Sensors vs operators An operator performs an action (run a query, call an API, move a file). A sensor waits for a condition to become true before letting downstream tasks proceed (e.g., "wait until this file lands in S3" or "wait until an upstream DAG finishes").

4. Scheduling a pipeline dependent on another team's pipeline Use an `ExternalTaskSensor` (waits on another DAG's task completion) or a data-driven trigger (e.g., a sensor watching for a "success" marker file/table in the upstream system) rather than just guessing a safe time buffer — time-based guessing is fragile and breaks when the upstream pipeline runs late.

ROUND 3: Cloud (AWS)

1. Pipeline design using S3, Glue, Redshift/Snowflake Raw files land in S3 (raw zone) → Glue crawlers/jobs clean and catalog the data → transformed data lands in a curated S3 zone or directly loads into Redshift/Snowflake → BI tools query the warehouse. Glue Data Catalog acts as the metadata layer so Athena/Redshift Spectrum can query S3 directly too if needed.

2. S3 storage classes Standard: frequent access, higher cost. Infrequent Access (IA): cheaper storage, retrieval fee, for data accessed occasionally. Glacier: very cheap, for long-term archival where retrieval can take minutes to hours — use for compliance/audit data banks must retain for years but rarely touch.

3. IAM Identity and Access Management — controls who (users, roles, services) can do what (permissions) on which resources. In pipelines, you'd assign a role to a Glue job or Lambda function with least-privilege access (only the specific S3 buckets/tables it needs), rather than broad admin access.

4. Partitioning data in S3 Organize files by columns commonly filtered on, e.g., `s3://bucket/table/year=2026/month=07/day=30/`. Query engines like Athena/Redshift Spectrum can then skip scanning irrelevant partitions entirely (partition pruning), dramatically cutting query cost and time.

5. Redshift vs Snowflake Both are cloud data warehouses. Redshift is tightly integrated with AWS, requires more manual tuning (distribution keys, sort keys, vacuum/analyze maintenance). Snowflake separates storage and compute more cleanly, requires less

manual tuning, and supports easier cross-cloud/multi-cluster scaling. Many banks are moving toward Snowflake for its lower operational overhead.

6. PII/data retention compliance in pipelines Encrypt sensitive fields at rest and in transit, mask/tokenize PII in non-production environments, implement column-level or row-level access controls, and build automated retention/deletion jobs to purge data past its legally required retention period. Maintain an audit trail of who accessed what data and when — banks are heavily audited on this.

ROUND 4: System Design Prompts — Approach for Each

1. Daily stock transaction pipeline for reporting Clarify: how many transactions/day, what's the reporting SLA (same day? next morning?), who consumes it (dashboards, regulators?). Then: batch ingestion from trading systems → staging in S3 → validate against expected row counts/reconciliation totals → transform into a star schema (fact: transactions, dimensions: security, trader, date) → load into warehouse → BI layer. Add monitoring on data completeness before the report is published.

2. Near-real-time fraud detection Clarify acceptable latency (seconds? sub-second?) and false-positive tolerance. Then: streaming ingestion (Kafka) from transaction events → stream processing (Spark Structured Streaming/Flink) applying rule-based or ML scoring → flagged transactions routed to an alerting system, all transactions also land in a data lake for later batch analysis/model retraining. Emphasize exactly-once processing so a transaction isn't double-flagged or missed.

3. Consolidating data from 5 bank branches Each branch likely has its own source system/format. Standardize via a common ingestion contract (schema, file format, drop location) per branch, land raw in S3 with branch-level partitioning, validate and conform schemas centrally, then merge into one warehouse fact table with a `branch_id` dimension. Handle each branch's pipeline independently so one branch's failure doesn't block the others.

4. Monitoring data quality across pipelines Centralized metadata/metrics store where every pipeline logs row counts, null rates, schema checksums, and freshness timestamps after each run. A monitoring dashboard/alerting layer (e.g., built on top of Great Expectations or custom checks) flags anomalies — compare against historical baselines, not just static thresholds, since "normal" volume varies by day of week/month-end.

5. Scaling from 1M to 100M records/day Move from single-node processing to distributed (Spark/EMR), reconsider file formats (columnar Parquet, proper partitioning), evaluate whether batch windows still meet SLAs or streaming becomes necessary, and check whether the warehouse's compute (Redshift/Snowflake warehouse size) needs to scale up. Also revisit whether the same orchestration tool and monitoring approach still work at 100x scale, or introduces new bottlenecks (e.g., too many small files — "small file problem").

ROUND 5: Behavioral — Sample Answers (Fill in your own specifics)

1. "Walk me through your transition from testing to data engineering — why this move?"

Sample structure: "In manual testing, I spent most of my time validating data flows — checking that what came out of a system matched what should have come out, tracing discrepancies back to root causes. I realized I was already doing informal data engineering work: verifying pipelines, checking data integrity, working with SQL daily. That pulled me toward formally building the pipelines rather than just testing them. Over the last [X months], I've built [mention your flagship project] to make that transition concrete rather than just theoretical."

2. "Tell me about a time you found a critical bug/data issue." Use STAR: Situation (what system/context), Task (what you were validating), Action (how you traced it — specific SQL queries, log analysis, cross-referencing source vs target), Result (impact — prevented incorrect reporting, saved rework, etc.). Quantify if possible ("caught a discrepancy affecting ~2,000 records before release").

3. "Describe a situation where you had to learn a new technology quickly." Pick a genuine example — e.g., learning Python/SQL/Airflow/Spark for your transition. Structure: what forced the urgency, how you approached learning (structured plan, hands-on project vs just reading docs), and the outcome (built X, applied it in Y context).

4. "Tell me about a time you disagreed with a teammate or manager." Focus on the resolution process, not who was "right." Show you raised the disagreement respectfully, backed it with data/reasoning, and were open to being wrong. Avoid stories where you "won" — interviewers want collaboration signals, not conflict-winning signals.

5. "How do you handle tight deadlines with incomplete requirements?" Emphasize: clarifying assumptions upfront, communicating early if requirements are unclear rather than guessing silently, delivering an MVP/incremental version, and flagging risks transparently rather than overpromising.

6. "Describe a project you're proud of — your specific contribution." Use your flagship DE project. Be very specific about *your* individual contribution (not "we built"), the technical choices you made and why, and one obstacle you overcame.

7. "How do you ensure data accuracy/quality?" Draw directly from testing background: systematic validation (row counts, boundary cases, reconciliation between source and target), not assuming happy-path data, and building repeatable checks rather than one-off manual verification.

8. "Tell me about a mistake you made — what did you learn?" Pick a real, moderate-stakes mistake (not catastrophic, not trivial). Show the fix and, more importantly, the process change you made afterward so it wouldn't repeat — interviewers care more about the learning than the mistake itself.

9. "Where do you see yourself in 3 years?" Something honest and growth-oriented: deepening technical expertise in data engineering (e.g., moving toward more

architecture/design ownership), possibly specializing in areas like streaming systems or data platform work, while contributing to a team solving real business problems.

10. "Why JPMorgan / why this team?" Research the specific team if possible (search their tech blog/engineering posts before the interview). Generic version: interest in working on data at scale within a regulated, high-stakes environment where data quality and reliability genuinely matter, plus the chance to work with a mature tech stack and experienced engineers to grow quickly.

ROUND 6: HR Round — How to Handle

1. "Do you have other offers?" Be honest but strategic — if you do, mention it factually without bluffing. If you don't, it's fine to say you're focused on finding the right fit rather than fabricating leverage.

2. "What are your salary expectations?" Research the market band for Associate/Junior Data Engineer roles in your city beforehand (Glassdoor, Levels.fyi, AmbitionBox for India). Give a range, not a single number, and anchor it to market data rather than just your current salary — testers are often underpaid relative to DE roles, so don't anchor too low based on your current pay.

3. Notice period / location / work mode Answer factually and confirm flexibility where you have it — banks often have fixed onboarding timelines, so clarity here helps you.

Practice tip: don't just memorize these — pick 3-4 questions per section and answer them out loud, unscripted, timing yourself to under 2 minutes each. That's the actual skill being tested, not recall.